

FRED TALKS

26th April 2026

CONTENTS

Liar's Valuation

NIKUNJ KOTHARI · 3242 WORDS

Mike Acton's Expectations of Professional Software Engineers

ADAM JOHNSON · 2223 WORDS

Liar's Valuation

NIKUNJ KOTHARI · 26 APR 2026 · [SOURCE](#)

What early employees should know before signing

We have models solving International Math Olympiad problems, AI writing production code, diagnosing diseases, passing the bar.

Yet CEOs of the best startups still can't tell bookings from revenue from ARR at first glance. Neither can most GPs at venture funds. The journalists covering these companies are building headlines too fast to *actually* ask.

Last week, a friend asked me about their offer to a hot AI company's \$10B valuation. They had no idea what that number actually meant, how the round was structured, what the lead investor really paid. They saw the headline and assumed their equity was valuable.

It's 2021 again. And 2015 again. Same games, same victims, just different buzzwords.

Before I go further: this post is about understanding what you're signing up for, not about whether you should sign up. Some of the best career decisions are financially irrational. You might join a company because you believe in the mission, because the team is exceptional, because you'll learn things you can't learn anywhere else. I've made those choices myself.

But even irrational decisions deserve clear eyes. Your time is the only asset you can't make more of. You should know what you're trading it for.

If you're new to startups and terms like "409A" or "strike price" or "fully diluted" sound foreign, I put together a [primer on startup equity](#) that covers the basics. Read that first, then come back here for how those mechanics get manipulated.

Part 1: Revenue

Let's start with definitions because most people, including many investors, use these terms interchangeably when they mean completely different things.

Bookings are promises. A customer signs a contract saying they'll pay you \$120,000 over three years. You've "booked" \$120,000. But you haven't collected it yet. They might pay monthly. They might churn in year two. The booking is a commitment, not cash in hand. Revenue recognition and cash flow are not the same thing, nor do they have to be.

Revenue is what you've actually earned. Under accounting rules, you recognize revenue as you deliver the service. That \$120,000 three-year contract becomes \$40,000 per year, recognized as you provide the product.

ARR is Annual Recurring Revenue, and it used to mean something specific. Subscription revenue that repeats. A customer paying \$1,000 per month represents \$12,000 in ARR. The word "recurring" was the whole point. It implied the revenue would continue because the customer was subscribed, not because they were making one-off purchases.

Investors paid high multiples for real ARR because the future was predictable. A company with \$10M ARR, 90% retention, and 75% gross margins might be worth 15-20x that revenue. The subscription persists. Customers expand over time. You can model it.

That definition is now dead.

The new formula: take last month's revenue, multiply by 12, call it ARR.

Doesn't matter if it's transactional revenue. Doesn't matter if margins are 15%. Doesn't matter if customers buy once and never return. Multiply by 12, call it ARR, and suddenly you're being compared to Salesforce. In egregious cases, doesn't matter if it's marketplace pass-through or GMV.

Transaction businesses now call themselves ARR companies.

Think about a full body MRI scan company. You pay \$3,000 for a scan. You get one scan. Maybe you'll get another in three years if you're diligent about preventive health. There's no subscription. Nothing recurring about it. It's a transaction, like buying a car.

But if that company did \$5M in scans last month, they'll report "\$60M ARR." One prominent VC firm's annual letter called a diagnostic imaging company an "ARR company." This blew my mind. These are healthcare services businesses with margins around 20% and no built-in retention mechanics. Public market multiples for businesses like this are 2-4x revenue, not the 15-20x that software companies command.

By calling it ARR, they get compared to software companies. They claim "fastest to \$100M ARR." They raise at software multiples while running completely different economics.

Data labeling companies do this. Marketplaces taking a cut of transactions do this.

Finance companies with transaction revenue do this too.

Some of the hottest fintech companies report massive ARR numbers. Dig into the composition and 80-90% of that “ARR” is transaction revenue from card swipes and payment processing. Not subscriptions. Not recurring in any meaningful sense.

If customers swipe less next month, the “ARR” drops. When recession hits and spending declines, this “ARR” craters. One well-known fintech’s engineering blog celebrated hitting “\$100M ARR” (you know the chart, “Years from \$1M to \$100M”) when the vast majority was interchange and transaction fees. Different risk profile. Different business model. Different appropriate multiple. On top of that, they compared annualized revenue run rate to annualized recurring revenue as if they were the same thing.

Hybrid models are even messier.

A vibe coding platform sells you a seat for \$20 per month. That’s subscription revenue. You also pay for usage: tokens, compute, API calls. That’s consumption revenue. These two types of revenue have completely different economics, but companies blend them together.

Here’s the math. Say a company has 1,000 customers paying \$20 per month for seats. That’s \$20,000 in monthly subscription revenue, or \$240,000 annually. Those same customers collectively spend \$200,000 per month on usage, which is \$2.4M annually. Total annual revenue is \$2.64M.

The company reports “\$2.6M ARR” as a single number.

But the subscription piece and the usage piece are fundamentally different. The subscription revenue is predictable with decent margins and probably good retention. The usage revenue swings wildly month to month, and margins are often brutal because you’re paying OpenAI or Anthropic for API calls and marking them up 20%.

Blending them into one ARR number hides everything important. A business that’s 90% usage-based gets compared to a business that’s 90% subscription-based as if they’re equivalent. They’re not.

Creative counting is the worst offender.

Free trials counted as ARR by assuming they’ll all convert. They won’t.

Monthly customers multiplied by 12 as “ARR” even though they signed month-to-month because they’re not sure about the product yet. That’s not annual recurring revenue. That’s monthly-maybe-recurring revenue.

Pilots counted as ARR before conversion. Verbal commitments treated as signed contracts. “Pipeline ARR” presented as if it were actual revenue.

I’ve seen all of these from companies that raised real money from sophisticated investors.

What this means for your equity:

When a company tells you they’re at “\$50M ARR” and growing 3x year over year, they’re using that number to justify their valuation. That valuation justifies your equity grant. Your equity grant is supposed to justify the risk you’re taking by joining.

But that \$50M might actually be \$30M in low-margin transactions that won’t repeat, \$15M in usage revenue that swings 40% month over month, and \$5M in actual recurring subscriptions.

The company presents one number. The economics underneath are three completely different businesses. Your equity is priced off the headline, not the reality.

Questions to ask before joining:

Always follow the money. Cash flow. What’s coming in from customers and what’s going out.

For traditional subscription companies: What percentage is annual contracts versus monthly? What’s net revenue retention? Are existing customers spending more over time or less? It should be over 100% for healthy software businesses. What’s gross margin?

For hybrid companies: Can you break out subscription versus usage revenue? How exactly is ARR calculated? What are margins by revenue type? What does retention look like in 12-month cohorts, not 3-month cohorts?

Most companies won’t share all of this. That tells you something. The ones confident in their numbers will explain because they’re proud of them.

At every startup I joined, I sat with whoever handled finance and refused to sign until I understood the real economics. Not paranoia. My time is worth more than optimistic storytelling.

Part 2: Valuation

Public companies have one price. Everyone buys and sells at the same number. You can disagree with whether the multiple is justified, but you can’t dispute what the price actually is.

Private markets don't work this way anymore.

How it's supposed to work:

A company raises a seed round. \$3M at a \$15M valuation cap. Every investor, whether it's the lead fund, the angels, or the operators writing small checks, participates at the same terms.

A company raises a Series A. \$10M at a \$50M post-money valuation. The lead writes \$8M, others fill in \$2M. Everyone pays the same price per share.

Your equity grant is based on that valuation. Your strike price reflects that price. Everyone's playing the same game with the same numbers.

How it actually works in 2025:

Tiered rounds where different investors pay wildly different prices, but only the highest price gets announced.

Venture funds have ownership targets. They need to own a meaningful percentage of winners to generate returns. At seed, a fund might want 15-20% ownership. At Series A, 15-20%. At growth stages, 10-15%. These targets are non-negotiable for serious funds because ownership drives returns.

So here's the problem. A company wants to raise \$10M at a \$1B valuation. That's 1% ownership. No serious lead investor will write that check. The ownership is too small. The board seat isn't worth it.

But founders want the billion-dollar valuation. It's a recruiting tool. A press headline. Signals momentum.

So they structure tiered rounds.

The lead investor puts \$50M at a \$500M valuation. They get their 10% ownership. The economics work for them. They did the diligence. They negotiated the terms. They decided what the company was actually worth.

Then the company opens a second tranche at a \$1B cap. FOMO investors pile in. Tourists who want to say they own a piece of a "unicorn." Funds that missed the earlier allocation. Angels who don't care about ownership targets because they're writing small checks anyway.

Press release goes out: "Company raises \$80M at \$1B valuation."

The lead paid \$500M. The tourists paid \$1B. A 2x spread depending on when you showed up.

Your 409A valuation, which determines your strike price, which determines what you pay to exercise your options, lands somewhere in that range. Usually closer to the headline than to what the smart money actually paid.

Let me walk through why this hurts you.

Companies want to show a large total comp. Salary plus equity per year, the biggest number possible to compete with everyone else. It's a vicious cycle.

\$250K/yr salary plus \$500K/yr equity and they get to email you an offer letter saying \$750K/yr total comp.

The headline number from a press article flows to an internal blogpost flows to your offer.

Company raises two tranches. Tranche A is \$50M at \$500M post-money. That's the lead investor. Tranche B is \$25M at \$1B post-money. That's everyone else.

The lead investor owns 10% at \$500M valuation. They did the real diligence. They set the terms. They decided \$500M was the right price.

Your 409A comes in at \$700M, somewhere between the two prices. Your option grant is 10,000 shares at a \$7.00 strike price.

Now here's the problem. For your options to be worth real money, the company needs to exit well above \$700M. But the most sophisticated investor in the deal valued it at \$500M. You're already paying a 40% premium over what smart money thought it was worth.

If the company exits at \$600M, the lead investor makes money. They paid \$500M. You lose money because your strike price was \$7.00 and the exit price might be \$6.00 per share. Same outcome, different results, because you bought at different prices.

Growth rate is the only thing that matters at high multiples.

Here's where I need to be honest. Some of these valuations will look cheap in hindsight.

In 2021, some software companies traded at 50-100x revenue. People called it a bubble. Those multiples crashed 70-80%.

In 2025, AI companies are raising at 200-500x revenue. People call it visionary.

OpenAI felt absurdly overvalued at \$10B. It's now worth 50x that. A company growing 4x annually will outrun almost any valuation if that growth sustains. The math that looks insane today becomes obvious in retrospect.

Growth rate is the only variable that matters at these multiples. Not the current revenue. Not the current multiple. The growth rate.

But here's the question you should actually ask: **How many months of perfect execution do you think this company needs to grow into this valuation?**

A company with \$2M revenue raising at \$1B valuation is trading at 500x. For that to become a reasonable 10x multiple, they need to hit \$100M in revenue. If they're growing 3x annually, that's about three years of flawless execution. No stumbles. No market shifts. No competition eating their lunch. Three years where everything goes right.

What's the probability of three years of perfect execution? Now you're asking the right question.

Most companies at 500x multiples won't grow into them. Some will grow past them. The ones that do will make everyone who dismissed them look stupid.

When evaluating a high-multiple company, focus on the runway to reasonableness. What's the month-over-month growth? What does the pipeline look like? How are cohorts retaining and expanding? How many months of perfect execution before this multiple makes sense? If the answer is 18 months and you believe in the team, maybe. If the answer is five years, you're buying a lottery ticket.

How to spot tiered rounds:

When you see a valuation that seems disconnected from reality and the round size seems small relative to that valuation, it was almost certainly tiered. A good litmus test is 10% ownership.

\$50M raised at a \$1B valuation. That's 5% ownership. No lead fund would accept that. The round was structured with multiple prices.

If you see a \$10B valuation for a company with \$30M ARR, that's 333x revenue. Either someone's lost their mind or there are multiple tranches baked in.

Questions to ask about your equity:

About the round structure: Was it raised at a single price or in tranches? What did the lead pay versus other participants? What percentage of the round came in at the headline valuation?

About your negotiating position: Given valuation risk, can I get more shares? Can I take more cash and less equity? Is early exercise available?

About your specific grant: What's the current 409A valuation? What's my strike price? What's my grant as a percentage of fully diluted shares, including all options and warrants, not just shares outstanding?

The company may not answer all of these. Watch how they react when you ask. Defensiveness tells you more than the answers would.

Part 3: This Keeps Happening

2000: Revenue recognition games. Companies booked revenue they'd never collect. Round-trip transactions inflated sales. Employees held options through the crash and watched them go to zero.

2008: CDOs sliced into tranches with different risk profiles. The sophisticated players understood exactly what they were buying. Goldman knew the CDOs were garbage. They packaged them anyway, sold them to German banks and pension funds, then shorted the same products. When the music stopped, Goldman had already exited. The German banks held the bag. Complexity wasn't a bug. It was the feature that let insiders profit while outsiders got destroyed. Everyone else held AAA-rated securities that turned out to be worthless.

Nobody went to jail. The game continued.

2021: "ARR" that included free trials, pilots, verbal commitments. Valuations at 100x revenue justified by zero interest rates. When rates rose, the music stopped. Public multiples crashed 70-80%. Private markdowns followed.

Employees who joined at peak valuations watched their equity evaporate. Some had exercised their options at high strike prices, owed taxes on paper gains through Alternative Minimum Tax, and then watched the stock become worthless. They owed the IRS money on gains that no longer existed and couldn't afford to pay because the shares they'd bought were now worth nothing. That's not an abstract risk. I know people this happened to.

The founders made their choices and live with the consequences. The investors had diversified portfolios across dozens of companies. Some won, some lost, the portfolio survived. The employees had concentrated positions in single companies and often no understanding of what they'd actually bought.

2025: Different year, identical playbook.

Companies report “ARR” that isn’t recurring. Valuations at 200-500x revenue. Tiered rounds where headlines diverge from reality. Journalists report the numbers without asking what they mean.

I’m not worried about investors. Their portfolios diversify. They’ll be fine.

I’m not worried about founders. They made their choices. They’re adults.

I’m worried about the engineer comparing offers who thinks ARR means ARR. The designer joining a “unicorn” who doesn’t know the round was tiered. The PM taking equity over cash because the multiple seemed reasonable based on numbers constructed to mislead.

These are the people who get hurt every cycle. They keep getting hurt because nobody explains how the game actually works.

What To Actually Do

First, decide why you’re joining.

If you’re joining to learn, to break into an industry, to work with people who will make you better, to build something you believe in: those are valid reasons that have nothing to do with financial math. Some of the best career decisions are economically irrational. I’ve made them myself. The learning and relationships from my startup years were worth more than any equity outcome.

But even irrational decisions deserve understanding. Know what you’re signing up for. Know what the equity is probably worth. Then make the choice with clear eyes.

If financial outcome matters to you, do the work. Scratch that. Always do the work. It’s your time.

Ask the questions from this post. Watch how the company reacts. The best companies will explain their numbers because their numbers are real. They’ll walk you through how ARR is calculated. They’ll explain the round structure. They’re proud of their metrics because the metrics reflect a genuine business.

Companies playing games will deflect. They’ll tell you it’s “not standard” to share that information. They’ll reframe your questions. They’ll make you feel like you’re being difficult for asking.

That reaction is your answer.

Weight cash appropriately. If the equity is speculative, with high multiples, unclear revenue quality, and tiered round structures, weight cash more heavily in your decision. Cash is liquid. Cash is real. You can pay rent with cash. You can invest cash in index funds. You know exactly what cash is worth.

Run the math yourself. A \$1B valuation with 10,000 shares out of 100M fully diluted means you own 0.01%. For that equity to be worth \$1M without dilution, the company needs to exit at \$10B. What's the probability of a \$10B exit? What growth rate is required? What has to go right?

Remember base rates. Most startups fail. Of those that don't fail, most don't generate meaningful returns for employees. The outcomes that make headlines, the early Googlers, the early Facebookers, are extreme outliers.

This doesn't mean you shouldn't join startups. It means you should join with clear eyes about what you're actually buying. The mission might be worth it. The team might be worth it. The learning might be worth it. Just don't confuse those things with the equity being worth it.

Thanks to [redacted] unicorn founders, [Ethan Ding](#), [Jake Cooper](#), [Mike Vernal](#) and others for providing comments and additions to this essay!

Subscribe to Balancing Act

By Nikunj Kothari

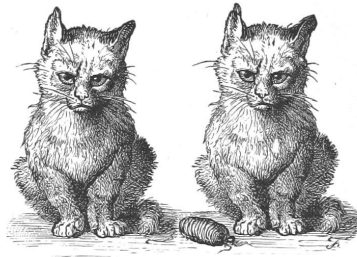
Candid thoughts about investing, operating and life.

By subscribing, I agree to Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).

[1 Like](#)·

Mike Acton's Expectations of Professional Software Engineers

ADAM JOHNSON · 26 APR 2026 · [SOURCE](#)



grumpy cats ranting

In a 2019 talk/rant titled [“Everyone Watching This Is Fired”](#), games industry veteran Mike Acton rattled off a sample of 50 things he expects of developers he works with. The title refers to his tongue-in-cheek suggestion that anyone who doesn't meet all these requirements would be immediately fired.

Although his sense of humour isn't for everyone, the suggestions are valuable. The list forms a baseline for software engineers to compare themselves against, and it's not very specific to the games industry.

I couldn't find a textual version, so I've written up the list below, with my own expansions of each point and some further quotes from Mike. I hope this list inspires you to improve as an engineer, as it has me.

1. I can articulate precisely what problem I am trying to solve.

It's all too easy to get stuck in the weeds and lose track of why you're doing what you're doing. Keep top of mind what the actual end goal is do, and you might spot an alternative path.

2. I have articulated precisely what problem I am trying to solve.

Communicate the problem “out loud” to other team members, your product manager, etc.

3. I have confirmed that someone else can articulate what problem I am trying to solve.

Communication! Ensure your team is all on the same page. Make sure your understanding of the problem is complete.

4. I can articulate why my problem is important to solve.

If you solve the problem you're working on, who benefits, and how much?

5. I can articulate how much my problem is worth solving.

If you say it's worth "as long as it takes"... Mike does not have friendly words for you. For *any* problem there's a maximum amount of time and effort worth investing in solving it. At least have some idea of the upper bound.

6. I have a Plan B in case my solution to my current problem doesn't work.

Imagine you're days or hours before the deadline, and you can tell that completing Plan A will be impossible. What do you do instead? Maybe you have a simplified algorithm, or you can disable a certain subsystem. Have more than one plan.

7. I have already implemented my Plan B in case my solution to my current problem doesn't work.

Mitigate risk by writing the backup version first. This means you always have a safety net and you can learn more about the problem space in order to iterate on Plan A.

8. I can articulate the steps required to solve my current problem.

Programming only works when you break down problems into manageable chunks. Have a sketch of the steps to the end state before you begin.

9. I can clearly articulate unknowns and risks associated with my current problem.

There are always going to be things you don't know. You should know where they are in your plan, so you can manage them.

10. I have not thought or said "I can just make up the time" without immediately talking to someone.

Say it's Wednesday, you have a project due on Friday, and you get some new task dropped on your lap. You think "I'll do the new thing now, and make up the time for the original task by Friday"... mistake! Communicate about the conflict on Wednesday. Your product manager will help manage the timing and risk.

11. I write a “framework” and have used it multiple times to actually solve a problem it was intended to solve.

If you’re writing a tool of some kind, you should verify it works in practice. Too often people create something in isolation and it doesn’t end up delivering in the real world.

(This is how Django came to be: from a real team making real websites, on deadlines!)

12. I can articulate what the test for completion of my current problem is.

If you don’t know when to stop, you might find yourself going down rabbit holes, chasing unimportant marginal gains.

13. I can articulate the hypothesis related to my problem and how I could falsify it.

If a hypothesis cannot be proven wrong, there’s no knowledge to be gained. As Karl Popper showed, science only works through falsification.

14. I can articulate the (various) latency requirements for my current problem.

Any time you write code, you should consider when the output data is required. Not every caller needs output data instantly, and nor do you have an unbounded amount of time to perform everything. At least get an idea of the sensible bounds.

15. I can articulate the (various) throughput requirements for my current problem.

How much data needs to come through the system? How many bytes, requests, or frames per second?

16. I can articulate the most common concrete use case of the system I am developing.

You should know what actual users of your system will actually be doing most of the time. Having a vague idea doesn’t help, since knowing which patterns are common informs which way to write a given piece of code.

17. I know the most common actual, real-life values of the data I am transforming.

Beyond use cases, you should know the data inside the system. For example, if your function works with integers, you’d probably write it quite differently if 99% of the values are 0.

18. I know the acceptable ranges of values of all the data I am transforming.

Computer systems always have limits. Know the ranges for the data types you're working with (and enforce them).

19. I can articulate what will happen when (somehow) data outside that range enters the system.

Murphy's Law says "anything that can go wrong will go wrong". Know how your system will behave in such cases, and handle such problems if necessary.

20. I can articulate a list of input data into my system roughly sorted by likelihood.

Have an idea of the space of possible data, what's most likely, second most likely, etc. Code appropriately, for example checking for common error conditions first.

21. I know the frequency of change of the actual, real-life values of the data I am transforming.

Reason about the frequency of change and figure out how often you'll want to calculate derived values.

22. I have (at least partially) read the (available) documentation for the hardware, platform, and tools I use most commonly.

Read the friendly manual! Go a step beyond day-to-day reference, and try reading the full documentation to gain a deep understanding.

(Jens Oliver Meiert calls reading the HTML specification [the Web Developer's Pilgrimage](#).)

23. I have sat and watched an actual user of my system.

Watching users can massively break shift your view of how your software works. Do it!

24. I know the slowest part of the users of my system's workflow with high confidence.

Any workflow has a bottleneck. Make sure you know what it is so you can focus efforts there, if need be.

25. I know what information users of my system will need to make effective use of the solution.

Think about what documentation or data users need to understand and use your solution.

26. I can articulate the finite set of hardware I am designing my solution to work for.

Software requires hardware. Know what hardware your program targets, such as:

- CPU architectures
- Minimum requirements for memory, CPU speed, and network bandwidth
- Input devices
- Output devices
- The environment the hardware runs in (e.g. data centre or living room)

27. I can articulate how that set of hardware specifically affects the design of my system.

If you're targeting low end devices, how do you ensure you don't exhaust memory? If some users don't have pointing devices, how do you accommodate them?

28. I have recently profiled the performance of my system.

If you're developing a local app, run profiling tools regularly to gain an idea of performance over time. With server based programs, you can install an APM (Application Performance Monitoring) tool in production and have continuous profiling data.

29. I have recently profiled memory usage of my system.

Make sure you aren't wasting memory.

30. I have used multiple different profiling methods to measure the performance of my system.

There's no perfect profiling tool, so know how to use more than one.

For example, some great Python profilers are cProfile, py-spy, Austin, Scalene, Fil, and memray. They all have different characteristics and complement each other.

31. I know how to significantly improve the performance of my system without changing the input/output interface of the system.

Do you know the next step to optimize your system? You don't have to do it right now, as it may not be worth it, but you should have an idea what you'd do next to make your code faster. For example, use a faster but less convenient data structure, or convert a hot function into a faster language (such as Python to C with Cython).

This should also guide you to designing interfaces that are optimizable in the first place. For example, don't commit to returning expensive-to-compute results immediately, but instead return a promise.

32. I know specifically how I can and will debug live release builds of my work when they fail.

Bugs are inevitable. You should know the tools that will let you work through those problems in production, such as logs, debuggers, or a live shell.

33. I know what data I am reading as part of my solution and where it comes from.

Know where the data comes from, in what format, and how you can read it.

34. I know how often I am reading data I do not need as part of my solution.

Data access is rarely optimal. You'll often be moving data that's not required for your solution, such as unnecessary fields or wrapper objects. If you don't know about this waste, you can't reason about whether it's worth the overhead or not.

35. I know what data I am writing as part of my solution and where it is used.

All output data is intended for use by a human or another program. Be organized enough to know what the downstream consumers of your output are.

36. I know how often I am writing data I do not need to as part of my solution.

Data output is also rarely optimal. Are you frequently writing out data that hasn't changed? Are you writing many fields when only one is used downstream? Again, know about it so you can reason about it.

37. I can articulate how all the data I use is laid out in memory.

Many programming languages and frameworks can handle memory for you, but that doesn't abdicate you of responsibility. Know how your tools lay out memory, so you can tell when another approach makes sense.

For example, in Python most objects are based on dictionaries, so you should have a solid understanding of how they work, and alternatives like slotted classes or arrays.

38. I never use the phrase “platform independent” when referring to my work.

Any system depends on many things below it. Know what they are.

39. I never use the phrase “future proof” when referring to my work.

Future-proofing is “100% a fool’s errand”. “You can’t pre-solve problems you have no information of.”

40. I can schedule my own time well.

“You’re an adult person, just use a calendar.”

41. I am vigilant about not wasting others’ time.

Don’t waste time asking questions that you can google in five seconds. But also don’t waste loads of time struggling for days alone when you could get help from a team member in minutes! Find the balance.

42. I actively seek constructive feedback and take it seriously.

Ask for feedback and do something about it.

43. I am not actively avoiding any uncomfortable (professional) conversations.

If there’s something wrong at work, don’t put off talking about it.

44. I am not actively avoiding any (professional) conflicts.

If you’ve noticed something is going wrong, whether technically or communication wise, get those conflicts out in the open. Letting them stew never helps.

45. I consistently interact with other professionals, professionally.

Be courteous and professional! Mike jokes about setting an incredibly low bar: no yelling, no hitting, ...

46. I can articulate what I believe others should expect from me.

Have a standard for yourself and be ready to tell your co-workers what it is.

47. I do not require multiple reminders to respond to a request or complete work.

“Waiting for someone else to poke you is not an effective way to get your job done.”

48. I pursue opportunities to return value to the commons (when appropriate).

All our work builds on top of the work of countless others. At some point, you’ll have opportunities to give back to the community at large. For example, talking at meetups, making open source contributions, or even just discussing topics with your team to boost everyone’s skills.

49. I actively work to bring value to the people I work with.

You’re part of a team, so work to help them.

50. I actively work to ensure under-represented voices are heard.

Don’t stand by leaving this to be someone else’s problem. Do something to make sure that minorities are heard. This might mean ensuring that the minority person at work gets a chance to speak, that your hiring process is unbiased, or that your website is accessible for users who rely on screen readers.

Fin

Let’s learn and grow together,

—Adam

One summary email a week, no spam, I pinky promise.

Related posts:

© 2022 All rights reserved.